

LegalEase

AI Powered Legal Document Generator

COMPLETE PROJECT REPORT + CODING

Project description, scenarios, prerequisites, workflow,
AI model selection, architecture, development environment,
document generator, editing, conclusion, and coding.

The project explanation and coding are combined into ONE PDF.

LegalEase

AI Powered Legal Document Generator

Project Description & Design Document

This document contains the project concept, scenarios, prerequisites, workflow, AI model selection, architecture, development environment, document generation process, document editing, and conclusion.

1. Project Description

Overview

LegalEase is an AI-powered legal document generator designed to help users create structured legal documents from simple user-provided information. The system collects relevant details through an easy-to-use interface, processes the information using an appropriate AI model, and produces a draft legal document in a clear and editable format.

Objective

The main objective is to reduce the effort required to prepare commonly used legal documents by combining a guided form-based workflow with AI-assisted document generation. The system is intended to improve consistency, speed, and usability while keeping the generated content editable for review.

Key Features

The proposed system includes user input collection, AI-assisted drafting, document template support, document generation, editing, preview, and export. The architecture can also be extended later with authentication, document history, multiple templates, and additional language support.

2. Project Scenarios

Scenario 1 — Agreement Generation

A user selects an agreement type, enters names, dates, terms, responsibilities, and other required information. LegalEase processes the inputs and generates a structured draft agreement.

Scenario 2 — Notice Generation

A user provides the recipient details, purpose of the notice, relevant dates, and required statements. The system prepares a formatted notice that can be reviewed and edited before use.

Scenario 3 — Document Editing

After generation, the user can modify clauses, names, dates, or other details in the generated document. The edited version can then be previewed and exported.

Scenario 4 — Reusable Templates

Frequently used document structures can be maintained as templates. The AI fills appropriate sections from the user's inputs while preserving the expected document structure.

3. Prerequisites

Software

A development environment such as Visual Studio Code, Python or Node.js depending on the selected implementation, a modern web browser, Git, and access to the chosen AI model/API.

Frontend Knowledge

Basic knowledge of HTML, CSS, JavaScript and a UI framework such as React is useful for building the form, preview screen, and editing interface.

Backend Knowledge

Backend development is required for validating user input, calling the AI model, applying templates, generating documents, and securely managing API credentials.

AI/API Requirements

The project requires an AI language model capable of structured text generation. API keys must be stored securely on the server side and should not be exposed in client-side code.

Document Generation

A document-generation library or service is required to convert the generated content into a downloadable format such as PDF or DOCX.

4. Project Workflow

Step 1 — User Input

The user chooses the required legal document type and enters the information requested by the application.

Step 2 — Validation

The application checks required fields, date formats, missing information, and basic input consistency before sending the request for generation.

Step 3 — Prompt Preparation

The backend converts the structured form data into a controlled prompt or structured request containing the selected document type, facts, and formatting requirements.

Step 4 — AI Generation

The selected AI model generates a draft using the provided information and the defined document structure.

Step 5 — Post-processing

The application formats headings, clauses, names, dates, numbering, and other document elements. Validation can also be applied to required sections.

Step 6 — Preview & Edit

The user reviews the generated document, makes corrections or changes, and confirms the final content.

Step 7 — Export

The final document is converted into the selected output format and made available for the user to save or download.

5. Research and Selection of the Appropriate AI Model

Selection Factors

The AI model should be evaluated based on document-generation quality, instruction following, structured output support, context length, latency, API reliability, cost, privacy requirements, and integration effort.

Expected Model Behavior

The model should follow a defined document structure, preserve user-provided facts, avoid inventing missing details, clearly separate placeholders from supplied information, and produce consistent clause formatting.

Recommended Architecture Approach

For a prototype, an API-based large language model can be integrated through a backend service. The application should use templates and controlled prompts rather than relying on free-form generation alone.

Important Limitation

LegalEase should be treated as a document-assistance system rather than a substitute for a qualified legal professional. Generated documents should be reviewed for accuracy, jurisdiction-specific requirements, and suitability before actual legal use.

6. Architectural Diagram

System Architecture

The proposed architecture follows a simple client–server–AI workflow. The frontend collects user information and displays the generated document. The backend validates inputs, prepares prompts, communicates with the AI service, and performs document processing. A database can optionally store users, templates, and document history.

Component	Responsibility
User / Frontend	Collect inputs, show preview, editing and export controls
Backend API	Validation, prompt preparation, orchestration and security
AI Model	Generate structured legal-document draft
Document Generator	Apply formatting and create PDF/DOCX
Database / Storage	Optional templates, users and document history

7. Development Environment Setup

Environment Setup

Create the project repository, configure the frontend and backend, install required dependencies, and create environment variables for configuration.

Suggested Project Layers

Frontend UI → Backend API → Validation & Prompt Builder → AI Model Service → Document Formatter/Generator → Storage/Export.

Security Setup

Keep API keys and server secrets in environment variables. Validate and sanitize user inputs. Avoid sending unnecessary personal information to external AI services, and apply access controls when documents are stored.

Testing Setup

Prepare sample inputs for different document types and test missing fields, unusual inputs, formatting issues, AI response failures, and export failures.

8. Exploring the LegalEase Program

User Interface

The main interface can provide a document-type selector, input form, Generate button, document preview, Edit option, and Export option.

Generation Process

When the user submits the form, the frontend sends structured data to the backend. The backend validates the data, calls the AI service, receives the draft, formats it, and returns the result to the frontend.

Error Handling

If required data is missing, the user should receive a clear validation message. If the AI service or document generator fails, the application should display a safe error message and allow the user to retry.

9. Document Generator and Editing

Document Generator

The generator converts the AI-produced structured content into a professional document. It can apply document title, section headings, numbering, spacing, signatures, dates, and other formatting rules.

Editing

The generated document should remain editable so users can correct factual information, replace placeholders, add or remove clauses, and make final formatting changes.

Preview and Export

A preview screen allows users to inspect the document before export. The final version can be exported as PDF or DOCX depending on the implementation.

Future Enhancements

Possible extensions include document versioning, saved templates, multilingual generation, user authentication, audit history, clause libraries, and improved validation workflows.

10. Conclusion

Summary

LegalEase provides a practical workflow for AI-assisted legal document generation. By combining structured user inputs, controlled AI generation, validation, templates, document formatting, and an editing stage, the project can create a useful prototype that is easy to demonstrate and extend.

Next Development Stage

The next stage is to implement the actual application code: frontend screens, backend APIs, AI integration, document generation, editing, and export. The coding section can be added to this project document after the core project-description PDF is finalized.

LegalEase

AI Powered Legal Document Generator — Coding

Starter full-stack project code for frontend, backend, AI integration point, validation, preview and PDF printing.

1. Project Structure

```
LegalEase/  
■■■ app.py  
■■■ requirements.txt  
■■■ .env  
■■■ templates/  
■ ■■■ index.html  
■■■ static/  
    ■■■ style.css  
    ■■■ script.js
```

2. Backend — app.py

```
from flask import Flask, render_template, request, jsonify  
from dotenv import load_dotenv  
import os  
  
load_dotenv()  
app = Flask(__name__)  
  
DOCUMENT_TYPES = {  
    "rental": "Rental Agreement",  
    "employment": "Employment Agreement",  
    "notice": "Legal Notice",  
    "nda": "Non-Disclosure Agreement"  
}  
  
def build_prompt(document_type, user_data):  
    name = DOCUMENT_TYPES.get(document_type, document_type)  
    return f"""  
Create a structured draft of a {name}.  
  
Use ONLY the facts supplied below.  
Do not invent names, dates, amounts, addresses, or facts.  
Use [REQUIRED INFORMATION] when something important is missing.  
  
USER INFORMATION:  
{user_data}  
  
Include:  
- Title  
- Parties  
- Relevant definitions  
- Main clauses  
- Terms and conditions  
- Signature section  
- Review notice  
""".strip()  
  
def generate_with_ai(prompt):  
    # Connect your selected AI provider here.  
    # Keep the API key on the server.  
    return (  
        "LEGAL DOCUMENT DRAFT\n\n"  
        + prompt  
        + "\n\n[AI provider response goes here]"  
    )  
  
@app.route("/")  
def home():  
    return render_template("index.html")  
  
@app.post("/generate")  
def generate():  
    data = request.get_json(silent=True) or {}  
    document_type = data.get("document_type", "").strip()  
    user_data = data.get("user_data", "").strip()  
  
    if not document_type:  
        return jsonify({"error": "Select a document type."}), 400  
  
    if not user_data:  
        return jsonify({"error": "Enter the required information."}), 400  
  
    prompt = build_prompt(document_type, user_data)  
    document = generate_with_ai(prompt)  
  
    return jsonify({  
        "document": document,
```

```
        "notice": "Review the draft for accuracy and applicable legal requirements."
    })

if __name__ == "__main__":
    app.run(debug=True)
```

3. Frontend — templates/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>LegalEase</title>
  <link rel="stylesheet" href="/static/style.css">
</head>
<body>
  <main class="container">
    <h1>LegalEase</h1>
    <p class="subtitle">AI Powered Legal Document Generator</p>

    <section class="card">
      <label>Document Type</label>
      <select id="documentType">
        <option value="rental">Rental Agreement</option>
        <option value="employment">Employment Agreement</option>
        <option value="notice">Legal Notice</option>
        <option value="nda">Non-Disclosure Agreement</option>
      </select>

      <label>Document Information</label>
      <textarea id="userData"
        placeholder="Enter names, dates, address, terms, amount, etc."></textarea>

      <button id="generateBtn">Generate Document</button>
      <p id="status"></p>
    </section>

    <section class="card">
      <h2>Document Preview / Edit</h2>
      <textarea id="result"></textarea>
      <button id="printBtn">Print / Save as PDF</button>
    </section>
  </main>
  <script src="/static/script.js"></script>
</body>
</html>
```

4. CSS — static/style.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f3f6fa;
  color: #1f2937;
}

.container {
  max-width: 900px;
  margin: 40px auto;
  padding: 20px;
}

h1, .subtitle {
  text-align: center;
}

.card {
  background: white;
  padding: 22px;
  margin: 20px 0;
  border-radius: 12px;
  box-shadow: 0 2px 10px rgba(0,0,0,.08);
}

label {
  display: block;
  margin: 12px 0 6px;
  font-weight: bold;
}

select, textarea {
  width: 100%;
  box-sizing: border-box;
  padding: 12px;
  border: 1px solid #ccd3dc;
```

```
    border-radius: 8px;
    font-size: 15px;
}

textarea {
    min-height: 180px;
    resize: vertical;
}

button {
    margin-top: 14px;
    padding: 12px 18px;
    border: 0;
    border-radius: 8px;
    cursor: pointer;
    font-weight: bold;
}
```

5. JavaScript — static/script.js

```
const generateBtn = document.getElementById("generateBtn");
const printBtn = document.getElementById("printBtn");
const resultBox = document.getElementById("result");
const statusBox = document.getElementById("status");

generateBtn.addEventListener("click", async () => {
  const documentType = document.getElementById("documentType").value;
  const userData = document.getElementById("userData").value.trim();

  if (!userData) {
    statusBox.textContent = "Please enter the required information.";
    return;
  }

  statusBox.textContent = "Generating...";
  generateBtn.disabled = true;

  try {
    const response = await fetch("/generate", {
      method: "POST",
      headers: {"Content-Type": "application/json"},
      body: JSON.stringify({
        document_type: documentType,
        user_data: userData
      })
    });

    const data = await response.json();

    if (!response.ok) {
      throw new Error(data.error || "Generation failed");
    }

    resultBox.value = data.document + "\n\nREVIEW NOTICE:\n" + data.notice;
    statusBox.textContent = "Document generated.";
  } catch (error) {
    statusBox.textContent = error.message;
  } finally {
    generateBtn.disabled = false;
  }
});

printBtn.addEventListener("click", () => {
  if (!resultBox.value.trim()) {
    alert("Generate a document first.");
    return;
  }

  const w = window.open("", "_blank");
  const safe = resultBox.value
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;");

  w.document.write(`
<html>
<head>
  <title>LegalEase Document</title>
  <style>
    body { font-family: Arial; margin: 40px; white-space: pre-wrap; line-height: 1.6; }
  </style>
</head>
<body>${safe}</body>
</html>
`);
  w.document.close();
  w.print();
});
```

6. requirements.txt

Flask
python-dotenv

7. .env

AI_API_KEY=your_api_key_here

Keep API keys private. The actual AI-provider request should be implemented inside `generate_with_ai()` according to the provider selected for your project.

8. Run Commands

```
cd LegalEase
python -m venv venv

# Windows
venv\Scripts\activate

# macOS/Linux
source venv/bin/activate

pip install -r requirements.txt
python app.py
```

9. Working Flow

User → Select document → Enter details → Validate → Build prompt → AI generation → Preview → Edit → Print/Save PDF.

10. Project Note

This is a starter implementation for the academic project. The AI-provider integration, database, authentication, production security, robust PDF/DOCX generation, and jurisdiction-specific legal validation can be added as the next development stage. Generated legal documents should be reviewed before real-world use.